



Optimizing LLM Batch Inference with Workload-Aware DVFS

Yiheng Tao¹, Yuping Fan², Michael Papka^{2, 1}, Zhiling Lan^{1, 2}

¹University of Illinois Chicago | ²Argonne National Laboratory



Why Batch Inference Matters

LLM systems research has largely focused on online serving, while batch inference remains underexplored.

Yet AI-for-Science and synthetic-data workflows increasingly rely on massive batch inference jobs.

Example Scale

ALCF Inference Endpoints

150,000

requests per batch file

Large request queues

Hours-days of inference

High GPU node-hour cost

Small gains matter

Large batch queues amplify small throughput and energy-efficiency gains into major system-level savings.

Online Serving Reacts; Batch Inference Can Plan

Batch inference differs from online serving because the workload is visible before execution.

Online Serving

r1 r2 r3 ?

Dynamic arrivals
Unknown future workload
Latency / fairness constraints
Reactive scheduling

Batch Inference

Batch file [r1, r2, r3, ..., rn]

VS

Known request queue
Input-length distribution visible
Throughput / energy / GPU-hour goals
Proactive planning

Advance workload knowledge enables proactive optimization.

Core Idea: Workload Knowledge Guides Frequency Planning

If the best GPU frequency depends on workload shape, then known batch composition can guide frequency planning before execution.

Inspect Batch

prompt lengths
request count

Known batch

Bucket by Length

short

medium

long

Plan Frequency

avoid over-frequency

boost when useful

Example planning rule

Short-prompt bucket

Lower prefill demand
Throughput may saturate early
Avoid peak frequency

VS

Long-prompt bucket

Higher prefill demand
More useful work per batch
Use higher frequency if beneficial

Same GPU, different batch shape -> different frequency decision

Evidence question: Does the sweet spot move with input length?

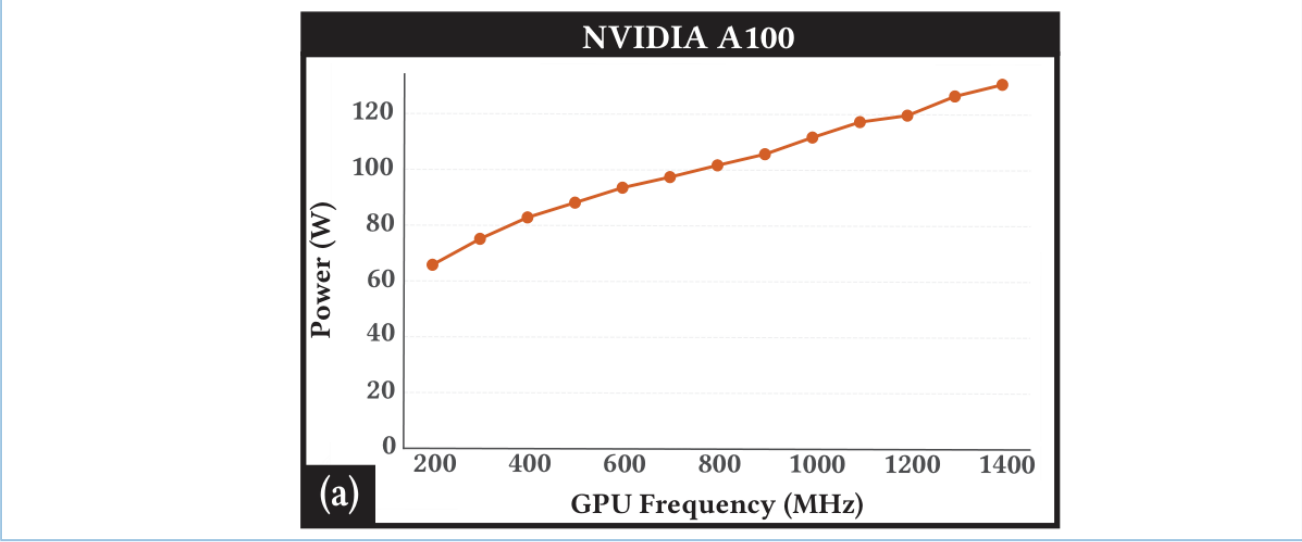
See Evidence 2

Evidence 1: Peak Frequency Is Not Always Optimal

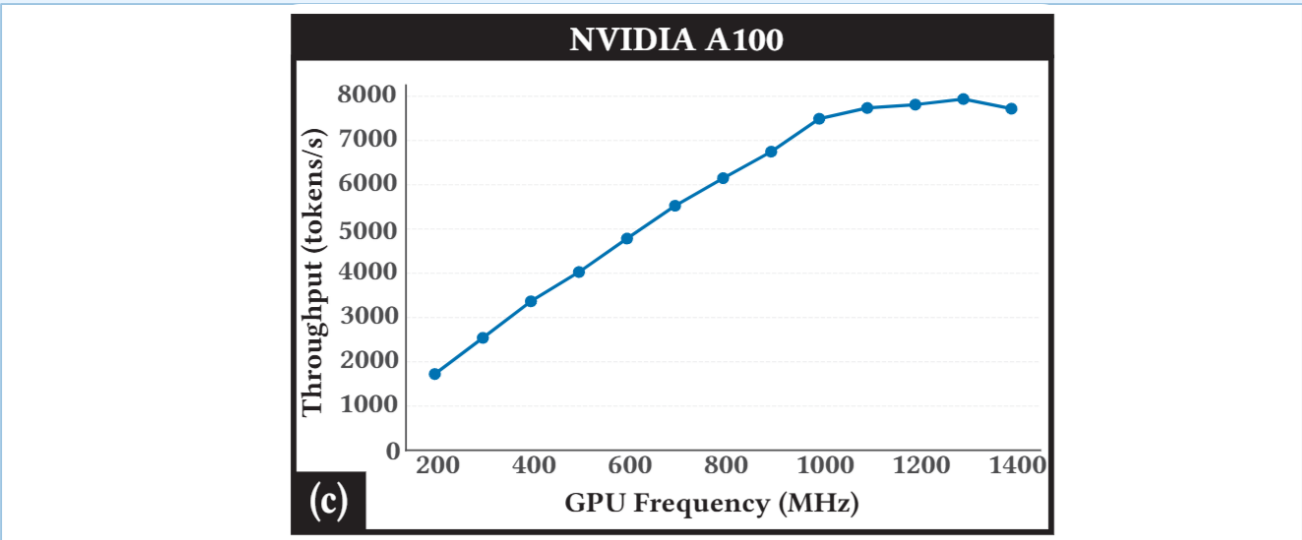
Power rises steadily with GPU frequency, while prefill throughput grows sub-linearly and eventually saturates.

The highest frequency can consume much more power for only marginal throughput gain.

Power vs Frequency



Throughput vs Frequency



Power keeps rising

Throughput saturates

-> Sweet spot below peak frequency

Maximum GPU frequency is not necessarily the most energy-efficient point.

Next: Does input length move the sweet spot? See Evidence 2.

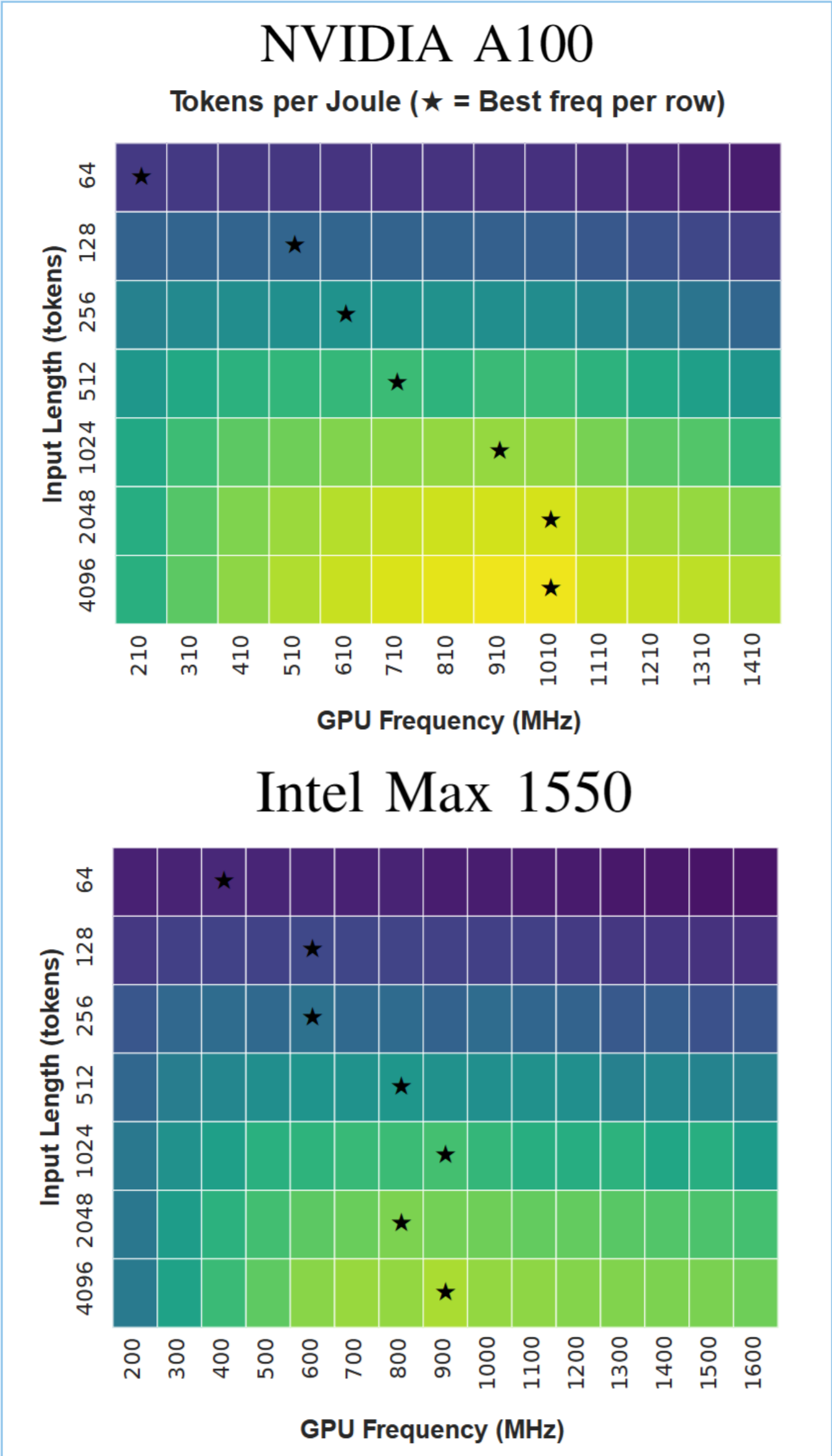
Evidence 2: Prefill Input Length Moves the Sweet Spot

In prefill, longer prompts increase workload intensity. The best tokens-per-joule frequency shifts with input length, making a single static frequency suboptimal for mixed batch workloads.

How to read

Rows = input lengths
Columns = GPU frequencies
Color = tokens/J ★ = best per row

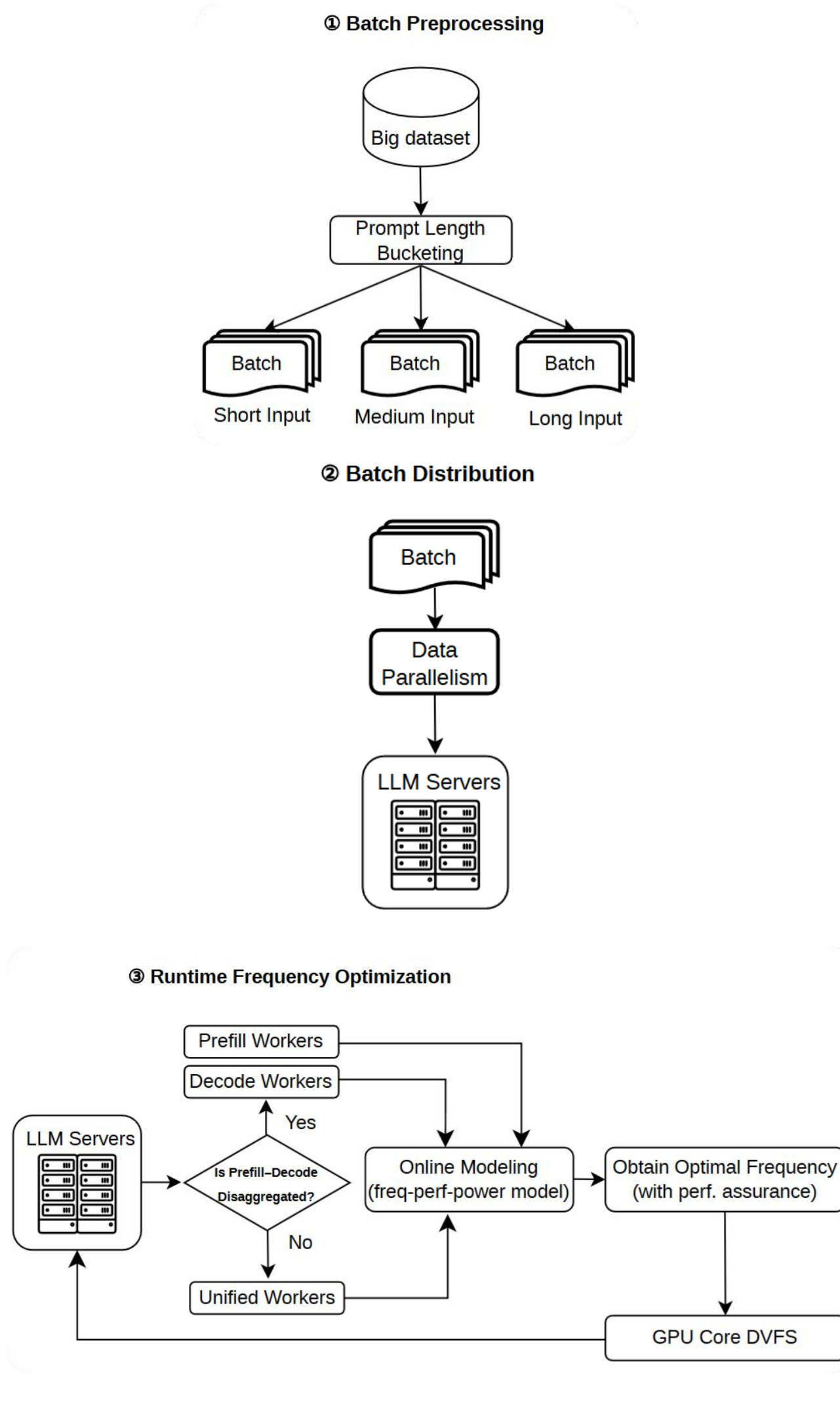
Longer inputs tend to shift the sweet spot upward



Because batch prompt lengths are known before execution, input length can guide frequency planning.

Proposed Direction: Batch-Aware Frequency Planning

Batch-Aware DVFS Pipeline



The pipeline converts known batch structure into frequency decisions: first bucket prompts by length, then distribute batches to workers, and finally select a GPU frequency target.

Known prompt lengths → workload intensity → frequency target

Method Components

Batch Analyzer: extracts request count and input-length distribution.

Length Bucketer: groups prompts into short / medium / long buckets.

Frequency Planner: maps workload intensity to a GPU frequency target with performance assurance.

Evaluation Targets & Expected Impact

What we measure

Tokens/J
Total GPU energy
Batch completion time
GPU node-hours

Expected impact

Avoid peak frequency when throughput saturates
Use higher frequency only for high-intensity buckets
Reduce energy cost and GPU node-hours for long-running batch jobs

Compare against max-frequency, static-frequency, and workload-agnostic policies.

Takeaway: Batch inference exposes workload knowledge in advance, enabling GPU frequency to be planned rather than guessed.